



NTNU

Norwegian University of Science and Technology
Department of Computer Science

TDT4230 Graphics and Visualization

Procedural Terrain Generation

Author

Simone D'Assisi

Year 2025/2026

Contents

1	Introduction	1
1.1	Technologies and Frameworks	1
2	Procedural Content Generation	2
2.1	Perlin Noise and Fractal Brownian Motion	2
2.2	Terrain Mesh Construction	2
2.3	Height-based Biome Classification	3
3	Vegetation and Hardware Instancing	3
3.1	Placement Logic and Environmental Constraints	4
3.2	The Coordinate Alignment Challenge	4
3.3	Hardware Instancing Implementation	4
4	Global Illumination Model	4
4.1	Diffuse Reflection	5
4.2	Specular Reflection	5
4.3	Ambient Integration	6
5	Dynamic Water Simulation	6
5.1	Gerstner Waves and Vertex Displacement	6
5.2	Water Shading and Optics	6
5.3	Procedural Foam and Atmospheric Integration	7
6	Atmospheric and Environmental Effects	7
6.1	Dynamic Day/Night Cycle	7
6.2	2.5D Volumetric Clouds	8
6.3	Exponential Distance Fog	8
6.4	Procedural Sun Rendering	9
7	Shader Architecture	10
8	Conclusion	10

1 Introduction

The objective of this project is the development of a three-dimensional virtual environment generated entirely through procedural techniques, utilizing modern graphics standards provided by OpenGL and the C++ programming language. The integration of mathematical algorithms for geometry and texture generation allows for overcoming the limitations of static assets, offering a potentially infinite and dynamic world characterized by a high degree of visual variability.

The technological core of the system lies in the application of Perlin Noise and Fractal Brownian Motion (FBM) functions, employed for terrain sculpting and the simulation of volumetric cloud formations. These algorithms enable the creation of natural transitions between distinct biomes—ranging from coastal areas to snow-capped peaks—faithfully simulating the morphology of real-world mountain ranges.

Beyond terrain mesh generation, the project addresses challenges regarding rendering optimization and environmental realism:

- **Instanced Rendering:** To populate the world with dense and complex vegetation without excessively straining GPU performance, hardware instancing was implemented. This technique allows for the drawing of thousands of 3D tree models with a significantly reduced number of draw calls.
- **Dynamic Lighting:** An integrated day/night cycle system was developed based on the rotation of a directional light source. This system dynamically affects terrain colors and cloud transparency.
- **Atmospheric Effects:** The project integrates exponential fog, enhancing depth perception and the overall immersiveness of the scene.

This report will detail the implementation choices, the architecture of the GLSL shaders, and the solutions adopted for coordinate space management, providing a comprehensive overview of creating complex digital worlds in real-time.

1.1 Technologies and Frameworks

The development of the procedural world was carried out using a C++ stack. The following libraries and standards were instrumental in building the simulation:

- **OpenGL 3.3 Core Profile:** The project utilizes the OpenGL 3.3 standard to ensure a modern shader-based pipeline. By using the *Core Profile*, all deprecated fixed-function features were avoided, forcing the use of Vertex Buffer Objects (VBO), Vertex Array Objects (VAO), and custom GLSL shaders for all rendering tasks.
- **GLFW:** This library is used to manage window creation and context handling. It provides a platform-independent API for handling the main loop.
- **Glad:** As an OpenGL loader, Glad was used to retrieve the addresses of OpenGL function pointers at runtime, enabling the application to interface correctly with the graphics drivers of the host system.

- **GLM (OpenGL Mathematics):** Since OpenGL does not provide its own math functions, GLM was integrated to handle all linear algebra operations. It was used extensively for calculating transformation matrices (Model, View, Projection) and interpolating colors with *mix* and *smoothstep*.
- **Tiny Object Loader:** This header-only library allowed for the parsing of external 3D models (OBJ format). It was used to load the tree mesh, which was then replicated throughout the world using hardware instancing.
- **GLSL (OpenGL Shading Language):** All visual logic—including procedural noise generation, dynamic lighting calculations, and atmospheric fog—was implemented in GLSL. This allows the heavy mathematical computations for every pixel to be offloaded from the CPU to the GPU.

2 Procedural Content Generation

The core of the simulation is the procedural generation of the terrain, which allows for the creation of complex landscapes without storing massive amounts of geometric data. This is achieved by using mathematical noise functions to determine the elevation and characteristics of the world.

2.1 Perlin Noise and Fractal Brownian Motion

Traditional random number generators produce white noise, which lacks the spatial continuity required for natural landscapes. To overcome this, the project implements **Perlin Noise**, a type of gradient noise that provides smooth transitions between values.

To achieve the "fractal" look of real mountains, where large slopes are covered with smaller rocks and details, we utilize **Fractal Brownian Motion (FBM)**. This technique involves stacking multiple layers of Perlin noise, known as *octaves*. Each successive octave has a higher frequency (more detail) and a lower amplitude (less influence). The resulting height H at a coordinate (x, z) is calculated as:

$$H(x, z) = \sum_{i=0}^{n-1} amplitude_i \cdot noise(frequency_i \cdot (x, z))$$

In my implementation, I used 6 octaves to balance visual detail with computational performance.

2.2 Terrain Mesh Construction

The terrain is represented as a heightfield grid. A custom **Terrain** class manages the generation of a mesh consisting of a grid of vertices. Each vertex contains:

- **Position (vec3):** The X and Z coordinates represent the grid horizontal position, while the Y coordinate is fetched from the FBM noise function.
- **Normal (vec3):** Calculated by sampling the heights of neighboring vertices. These normals are essential for the Phong lighting model and for determining where vegetation can grow.

The connectivity is defined using an **Element Buffer Object (EBO)**, which organizes the vertices into triangles. This approach significantly reduces the memory footprint compared to a raw triangle list, as vertices are shared between adjacent faces.

2.3 Height-based Biome Classification

Instead of using static textures, the *Fragment Shader* dynamically assigns colors based on the elevation (Y coordinate) of each fragment. This allows for a seamless transition between different environments:

Elevation Range	Biome	Visual Representation
$Y < 4.0$	Deep/Shallow Sea	Dark to light sand/blue tint
$4.0 \leq Y < 6.0$	Beach	Pale yellow sand
$6.0 \leq Y < 15.0$	Grassland	Green vegetation
$15.0 \leq Y < 25.0$	Rock/Mountain	Dark gray stone
$Y \geq 25.0$	Snow Peaks	Pure white snow

Table 1: Logic for altitude-based biome distribution.

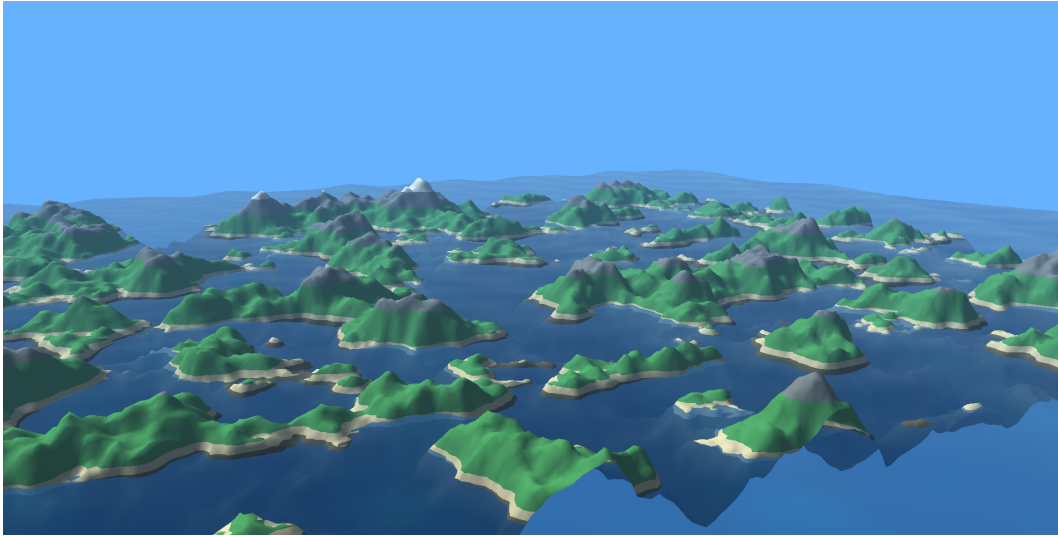


Figure 1: Procedurally generated terrain using Perlin Noise and Fractal Brownian Motion. Elevation values determine the formation of mountains, beaches, and oceans.

3 Vegetation and Hardware Instancing

A key challenge in creating an immersive procedural world is populating the environment with a high density of objects, such as trees, without compromising real-time performance. This section describes the logic used for object placement and the technical implementation of hardware instancing.

3.1 Placement Logic and Environmental Constraints

To ensure a natural distribution of vegetation, the placement of trees is not purely random but governed by environmental constraints derived from the terrain data. The algorithm iterates through the generated heightmap and selects suitable candidates based on:

- **Altitude Filtering:** Trees are only placed within the "Grassland" biome elevation range ($6.0 \leq Y < 15.0$), avoiding beaches, water, and snowy peaks.
- **Slope Constraint:** By sampling the vertex normals, the system ensures that trees only grow on relatively flat surfaces. If the vertical component of the normal (N_y) is below a certain threshold, the area is considered too steep for vegetation.

3.2 The Coordinate Alignment Challenge

A significant technical hurdle involved aligning the tree instances with the procedural terrain. Since the terrain is centered at the origin and scaled, a precise mapping was required to translate grid indices to world-space coordinates. I addressed this by calculating the model matrix for each instance as follows:

1. **Translation:** The instance is moved to $(x \cdot \text{scale}, \text{height}, z \cdot \text{scale})$, accounting for the terrain's global offset.
2. **Rotation:** A random rotation around the Y -axis is applied to break visual repetition.
3. **Scaling:** A slight random variation in scale is introduced to simulate natural growth diversity.

3.3 Hardware Instancing Implementation

Rendering thousands of individual tree models using standard draw calls would create a severe bottleneck in the CPU-GPU communication. To optimize this, I implemented **Hardware Instancing**.

Instead of sending the same mesh data multiple times, the tree geometry is uploaded to the GPU once. I then created a dedicated **Vertex Buffer Object (VBO)** containing the specific transformation matrices for all instances. By using the `glVertexAttribDivisor` function, I instructed OpenGL to advance the matrix attribute only once per instance rather than per vertex. This allows the entire forest to be rendered with a single call to `glDrawElementsInstanced`, drastically reducing CPU overhead and maximizing GPU throughput.

4 Global Illumination Model

To give volume and realism to the procedural geometry, the simulation employs a localized illumination model heavily inspired by the classic **Phong reflection model**. Since the primary light source is the sun (or moon), the system utilizes a single *Directional Light*. Unlike point lights, a directional light is assumed to be infinitely far away, meaning its light rays are strictly parallel across the entire scene and do not suffer from distance attenuation. The lighting calculations are performed per-fragment within the *Fragment Shaders* to ensure

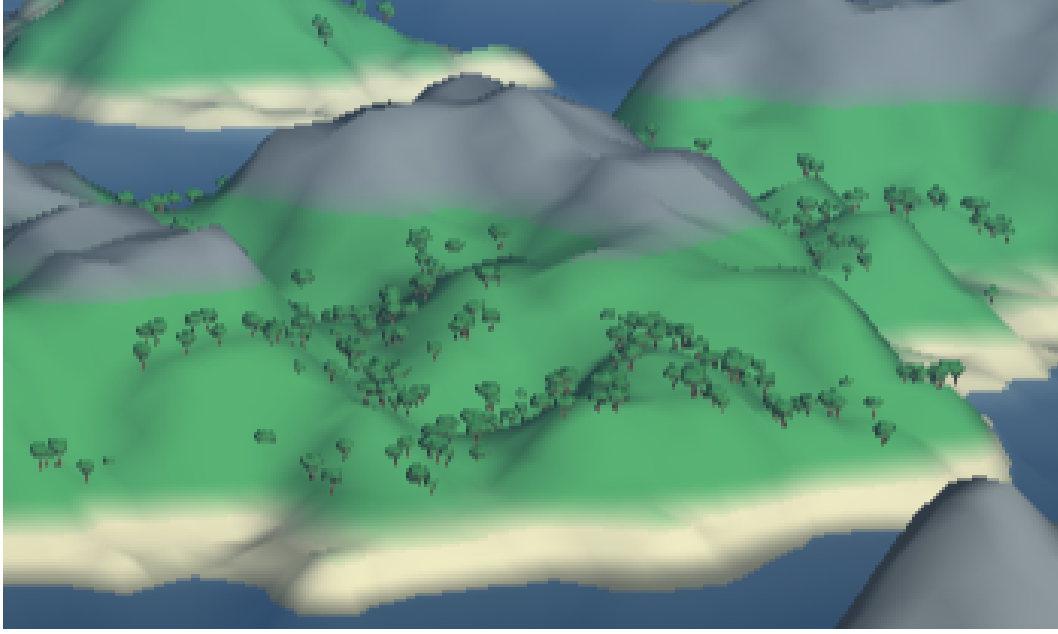


Figure 2: Dense vegetation distributed across the terrain. Trees are procedurally placed based on altitude filtering and terrain slope constraints.

smooth shading across the terrain and water surfaces. The final color of each pixel is determined by computing distinct optical components:

4.1 Diffuse Reflection

The diffuse component simulates the directional impact of the light on a rough, matte surface (Lambertian reflectance). It depends solely on the angle between the surface normal vector $\vec{N}$ and the inverse light direction vector $\vec{L}$. The diffuse intensity I_d is calculated using the dot product:

$$I_d = \max(\vec{N} \cdot \vec{L}, 0) \cdot C_{light}$$

where C_{light} is the dynamic color of the sun at the current time of day. This fundamental calculation ensures that slopes directly facing the sun are brightly lit, while opposite sides naturally fall into shadow.

4.2 Specular Reflection

To simulate the intense glare of the sun, particularly on the dynamic water surface, a specular highlight is computed. This component requires calculating the reflection vector $\vec{R}$ of the light ray around the surface normal, and comparing its alignment to the view direction $\vec{V}$ (the camera's position). The specular intensity I_s is given by:

$$I_s = (\max(\vec{V} \cdot \vec{R}, 0))^\alpha \cdot C_{light}$$

where α represents the shininess exponent. For the water surface, a very high exponent ($\alpha = 128.0$) was chosen to create the tight, bright spots of light characteristic of reflective

liquids. Furthermore, the specular color is artificially boosted to emphasize the wetness of the surface.

4.3 Ambient Integration

Pure Phong lighting often leaves unlit areas completely black, which is physically inaccurate due to global light scattering. To simulate ambient occlusion and skybox radiosity without the immense computational cost of ray tracing, the ambient light is dynamically linked to the sky's color. As the sun sets and the main light intensity approaches zero, the base color of the terrain and clouds is mixed directly with the ambient sky color, ensuring the environment remains softly illuminated at night.

5 Dynamic Water Simulation

To complement the procedural terrain, a dynamic water simulation was implemented using a custom shader-based approach. The water surface is not a static mesh but is continuously animated on the GPU to simulate the behavior of ocean waves and their complex interaction with light.

5.1 Gerstner Waves and Vertex Displacement

Instead of relying on simple sine waves, the vertex shader implements **Gerstner Waves**, which provide a much more realistic, steep-crested wave profile. The final surface is generated by summing three distinct directional waves, each characterized by its own steepness, wavelength, and speed parameters.

Unlike basic displacement mapping that only affects the vertical axis, the Gerstner wave algorithm modifies the X , Y , and Z coordinates. This moves the vertices in a circular motion, pushing the crests upward and pinching them together, while widening the troughs. To ensure accurate lighting across this continuously deforming surface, the normals cannot be pre-computed. Instead, they are calculated analytically inside the vertex shader. This is achieved by computing the tangent and bitangent vectors derived from the partial derivatives of the Gerstner wave function. The final accurate normal is obtained via the cross product of these two vectors.

5.2 Water Shading and Optics

The fragment shader handles the complex optical properties of the water surface, focusing on color interpolation, reflectivity, and dynamic transparency.

- **Depth-Based Color:** The base color of the water is interpolated between a dark blue for deep water and a lighter, cyan hue for shallow areas. This is calculated dynamically using the fragment's vertical position relative to the base `waterLevel`.
- **Specular Highlights:** To simulate the intense reflection of the sun on the water surface, a strong specular highlight with a high shininess exponent (128.0) is applied to the final color.

- **Fresnel Effect:** The reflectivity and transparency of water naturally vary depending on the viewing angle. A Fresnel factor is calculated to increase the reflection intensity and the overall opacity of the water when the surface is viewed at a grazing angle.

5.3 Procedural Foam and Atmospheric Integration

To further enhance realism, procedural foam is dynamically generated exclusively on the wave crests. The foam logic evaluates three distinct parameters:

- **Crest Factor:** It identifies the peaks of the waves by checking how much the surface normal deviates from the strict vertical axis ($1.0 - N_y$).
- **Height Factor:** The foam is restricted to appear only on waves that rise sufficiently above the base water level, utilizing the `smoothstep` function.
- **Animated Noise:** A pseudo-random noise function, animated over time (`uTime`), creates an organic, breaking pattern for the foam distribution.

Finally, the foam color is mixed with the water, and the entire surface seamlessly blends into the environment through the application of the global exponential fog. This ensures the water visually merges with the distant horizon without harsh geometric cutoffs.

6 Atmospheric and Environmental Effects

To create a believable and immersive atmosphere, I implemented dynamic systems that simulate the passage of time and the density of the air. These effects rely on the real-time coordination between C++ timing logic and GLSL fragment calculations.

6.1 Dynamic Day/Night Cycle

The environment features a continuous day/night cycle driven by a rotating directional light source. The sun's path is calculated using trigonometric functions of the elapsed time, following a circular trajectory across the sky. This system dynamically modifies the global aesthetic by interpolating between three primary color palettes:

- **Daylight:** Characterized by high-intensity light and a bright blue sky (RGB: 0.4, 0.7, 1.0).
- **Dawn/Dusk:** Featuring warm orange and reddish hues (RGB: 0.8, 0.4, 0.2) to simulate atmospheric scattering at low sun angles.
- **Night:** A low-intensity, dark blue ambient light (RGB: 0.08, 0.12, 0.25) that preserves visibility while establishing a nocturnal mood.

I utilized the `glm::smoothstep` function to ensure these transitions occur gradually, providing a smooth visual flow as the lighting conditions evolve.

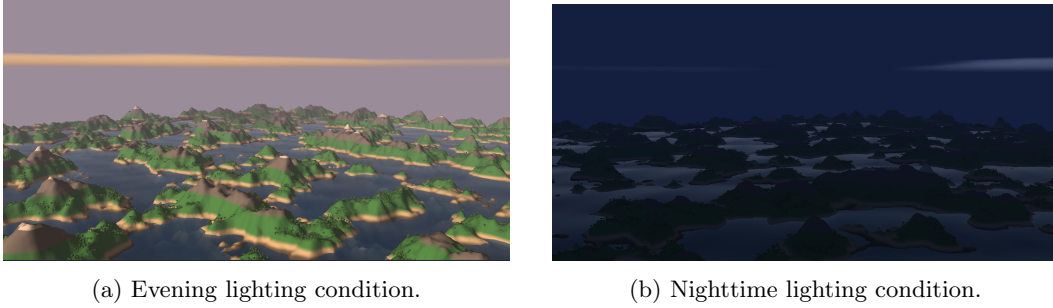


Figure 3: Dynamic day and night cycle driven by the rotation of a directional light source. The color of the sky and ambient lighting gradually transition between daylight and nighttime conditions.

6.2 2.5D Volumetric Clouds

Rather than using simple 2D textures, I developed a "2.5D" volumetric cloud system. This is achieved by rendering 15 horizontal quads stacked vertically at high altitudes (starting at $Y = 110.0$). Each layer samples a 2D Fractal Brownian Motion noise function, but with a specific coordinate offset and vertical thickness variation determined by the layer's fraction. By applying a `discard` command on fragments where the noise density is below a calculated threshold, I created "sculpted" cloud shapes that appear to have volume and depth when viewed from the terrain. To ensure visual consistency, these layers are rendered with depth writing disabled (`glDepthMask(GL_FALSE)`), allowing for proper alpha blending without artifacts.



Figure 4: Procedural cloud layers generated using fractal noise. Multiple semi-transparent layers create the appearance of volumetric clouds in the atmosphere.

6.3 Exponential Distance Fog

To enhance the sense of scale and seamlessly hide the boundaries of the procedurally generated grid, I implemented an **Exponential Fog** system across the fragment shaders. Unlike

linear fog, which can appear artificial and flat, exponential fog more accurately models the physical scattering of light through atmospheric particles over vast distances.

The fog calculation evaluates the Euclidean distance between the camera (**viewPos**) and the current pixel in world space (**FragPos**). The visibility factor, or *FogFactor*, is computed using the exponential decay formula:

$$\text{FogFactor} = \text{clamp}(\exp(-(\text{distance} \cdot \text{density})), 0.0, 1.0)$$

This factor is then used to linearly interpolate (**mix**) the rendered color of the object with the global fog color. A crucial aspect of this implementation is its tight integration with the dynamic day/night cycle: the **fogColor** uniform is continually updated to match the current **skyColor**. As a result, the terrain, water, and trees organically fade into the horizon, tinting correctly with the orange hues of dusk or the deep blues of the night.

6.4 Procedural Sun Rendering

To provide a visual focal point for the dynamic lighting system, a procedural sun was implemented. Instead of using a static skybox texture, the sun is rendered as a 3D sphere procedurally generated at initialization.

To create the illusion of a distant celestial body, the sun's model matrix is updated every frame so its position remains consistently far from the camera along the calculated light direction vector (**camPos** + **sunDir** * 800.0f). This technique ensures the sun remains unreachable, moving harmoniously with the player's view and naturally setting behind the procedural mountains.

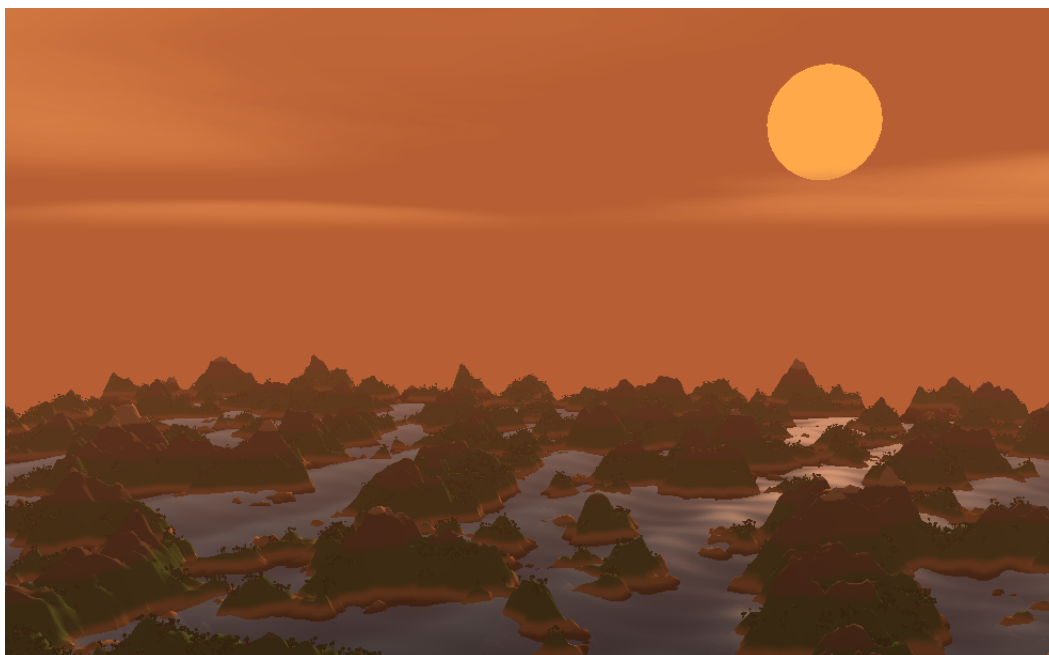


Figure 5: Procedural sun rendering during the sunset phase. The sun's position and color dynamically update based on the day/night cycle, seamlessly blending with the volumetric clouds and the exponential fog.

Furthermore, the sun’s base color dynamically interpolates between a bright white at noon and a deep orange at dawn and dusk, perfectly complementing the sky’s ambient color transitions. To maintain proper depth sorting, the sun sphere is rendered as an opaque object immediately before any transparent elements, ensuring realistic occlusion and blending with the volumetric clouds and water.

7 Shader Architecture

The rendering pipeline of the project is composed of several specialized GLSL shader programs. Each environmental component is rendered using a dedicated pair of vertex and fragment shaders, allowing the visual logic to remain modular and maintainable. The main shaders used in the project are:

- **Terrain Shader:** responsible for terrain lighting, height-based biome coloration, and exponential fog integration.
- **Water Shader:** performs vertex displacement using Gerstner waves and computes dynamic reflections, Fresnel effects, and procedural foam.
- **Tree Shader:** used for rendering vegetation through hardware instancing, applying per-fragment lighting and fog.
- **Cloud Shader:** generates procedural volumetric clouds using animated fractal noise.

This separation ensures that computationally expensive operations, such as wave deformation and procedural noise evaluation, are executed directly on the GPU, minimizing CPU overhead.

8 Conclusion

This project demonstrates how procedural generation techniques can be combined with modern GPU programming to create complex virtual environments in real time.

By integrating terrain generation, dynamic water simulation, hardware-instanced vegetation, atmospheric fog, and volumetric cloud layers, the system produces a cohesive and visually rich environment entirely driven by mathematical functions.

Future work could extend the system with more advanced rendering techniques such as shadow mapping, screen-space reflections for water surfaces, and physically based atmospheric scattering models.

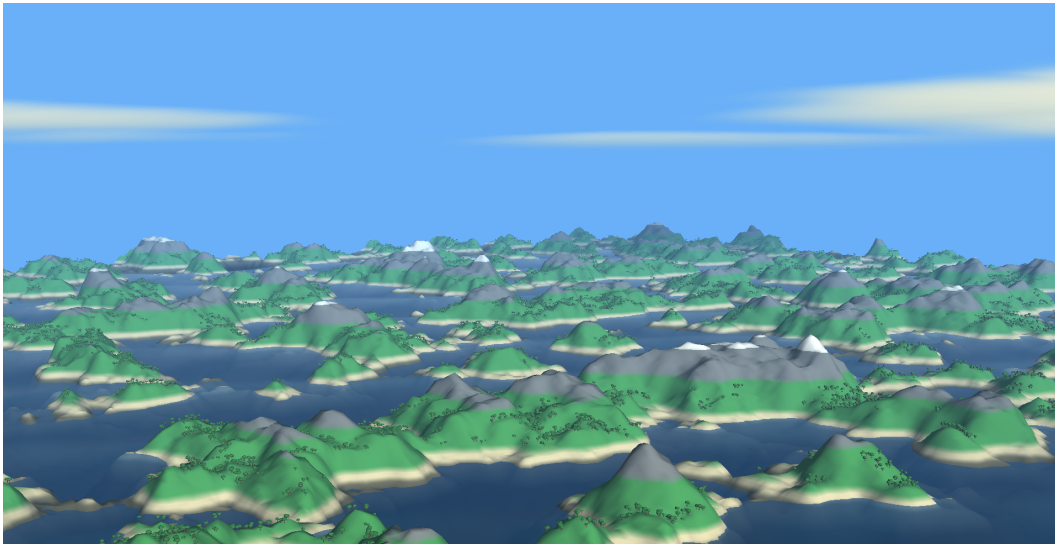


Figure 6: A screenshot of the completed project.